

AI Companion Backend Architecture & Prompt Guide

1. Project Goal

Build a production-ready backend using Spring Boot with JWT authentication, role-based access control, modular architecture, and AI-powered RAG system using PostgreSQL with pgvector.

2. Backend Package Structure

```
com.badri.companion
├── config/
├── common/
├── auth/
├── user/
├── content/
├── ai/
├── quiz/
└── infrastructure/
```

3. Auth Module

```
auth/
├── controller/AuthController
├── service/AuthService, TokenService
├── dto/LoginRequest, RegisterRequest, AuthResponse
├── security/JwtAuthenticationFilter, JwtTokenProvider,
│   CustomUserDetailsService, SecurityConfig
```

4. User Module

```
user/
├── entity/User, Role
├── repository/UserRepository, RoleRepository
└── service/UserService
```

5. Content Module

```
content/
├── entity/Category, SubCategory, Document, DocumentChunk
├── repository/CategoryRepository, DocumentRepository,
│   DocumentChunkRepository
├── service/CategoryService, DocumentService, IngestionService
└── controller/CategoryController, DocumentController
```

6. AI Module

```
ai/
├── service/EmbeddingService, VectorStoreService,
│   RetrievalService, ChatService, PromptService
└── config/AiConfig
```

7. Quiz Module

```
quiz/  
■■■ entity/Quiz, Question, Answer, Evaluation  
■■■ repository/QuizRepository, EvaluationRepository  
■■■ service/QuizService, EvaluationService  
■■■ controller/QuizController
```

8. Common Module

```
common/  
■■■ exception/GlobalExceptionHandler, CustomException  
■■■ dto/ApiResponse  
■■■ util/SecurityUtils
```

9. Infrastructure Module

```
infrastructure/  
■■■ persistence/PgVectorCustomRepository  
■■■ external/OpenAiClientWrapper
```

10. RAG Flow

1. Upload document 2. IngestionService splits content into chunks 3. EmbeddingService generates embeddings 4. Store in DocumentChunk (pgvector) 5. RetrievalService performs similarity search 6. ChatService calls LLM and returns response

11. Core Prompt Template

You are a senior backend architect.

Design a modular Spring Boot backend with:

- JWT authentication and RBAC
- PostgreSQL + pgvector
- RAG system (embedding, retrieval, LLM)

Focus on scalability, maintainability, and clean architecture.
Avoid microservices for now.